

who else is listening?

digitalisierung und programmierung · prof. dr. nicolas meseth

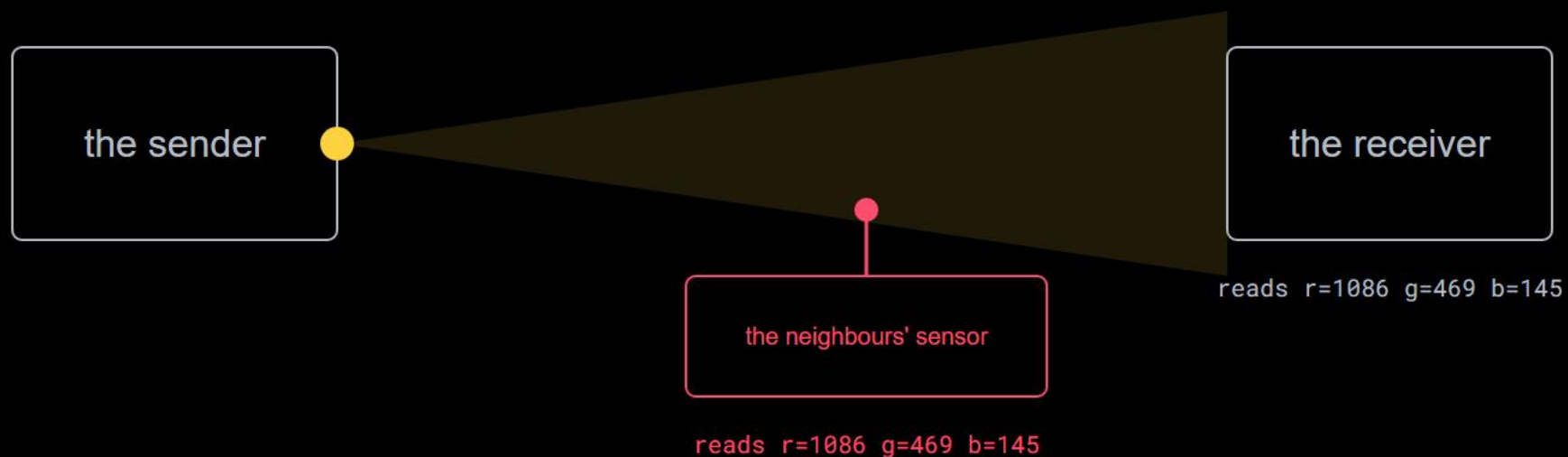
- 1 light is public
- 2 one gate, one key
- 3 how long is long enough
- 4 what a key does not do

part 1

light is public

[illegible]

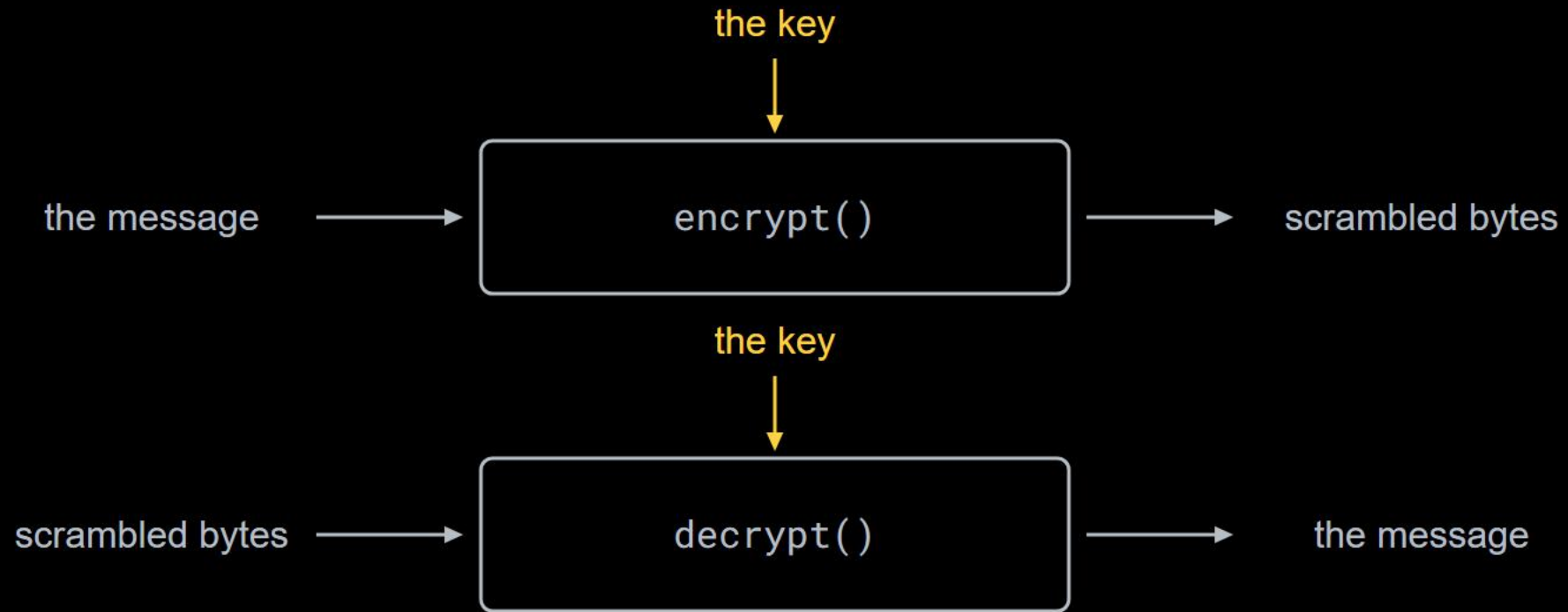
the third sensor



light spreads. whatever your sensor sees, a sensor next to it sees too, and you cannot tell.

whatever your sensor sees, the sensor next to it sees too.

encryption as a box, twice



the same key, known to two devices and to nobody else. that second input is the whole difference.

everyone may see the message. nobody but you may understand it.

part 2

one gate, one key

you have seen this gate



a 1 in the key flips the bit.
a 0 leaves it.

a	b	a xor b
0	0	0
0	1	1
1	0	1
1	1	0

logic and arithmetic: the toolbox, and the parity bit

xor: 1 when the two differ. that is all we need.

xor with a key byte

the byte for m 0110 1101 = 6d

where the key has a 1, the bit flips. the result looks like any old byte.

xor with a key byte

the byte for m 0110 1101 = 6d

the key 1010 1010 = aa

where the key has a 1, the bit flips. the result looks like any old byte.

xor with a key byte

the byte for m 0110 1101 = 6d

the key 1010 1010 = aa

sent 1100 0111 = c7 where the key has a 1, the bit flipped

where the key has a 1, the bit flips. the result looks like any old byte.

do it again, get it back

received 1 1 0 0 0 1 1 1 = c7

the same key 1 0 1 0 1 0 1 0 = aa

encrypt and decrypt are the same operation with the same key.

do it again, get it back

received	1 1 0 0	0 1 1 1	= c7
the same key	1 0 1 0	1 0 1 0	= aa
back	0 1 1 0	1 1 0 1	= 6d the same positions, flipped back: the m again

encrypt and decrypt are the same operation with the same key.

the key of all zeros

key 0000 0000: nothing flips.

the program runs, the transmission works, and you are sending plaintext without noticing. look at the bytes that actually go out.

what it looks like in the file

plain: one byte per character

6d 65 65 74 20 61 74 20 6e 6f 6f 6e meet at noon

key: lifi, written under the message as often as needed

6c 69 66 69 6c 69 66 69 6c 69 66 69 lifilifilifi

sent: plain xor key, byte for byte

01 0c 03 1d 4c 08 12 49 02 06 09 07 ...l..i...

12 bytes in, 12 bytes out. the same length, the same sending time, and not one byte anyone can read.

same length, same number of bytes, not one of them readable.

try it: scramble your own text

scramble: **a text** the parrot

the message

meet at noon

the key (repeated as often as needed)

lifi

try: meet at noon · key lifi a sentence · one-byte key the key of zeros

a key as long as the message

plain 6d 65 65 74 20 61 74 20 6e 6f 6f 6e

key 6c 69 66 69 6c 69 66 69 6c 69 66 69

sent 01 0c 03 1d 4c 08 12 49 02 06 09 07

...L..I...

decrypt with the same key

try all keys

12 bytes scrambled with a 4-byte key, repeated 3 times. same length as before, not one byte readable.

part 3

how long is long enough

your spec may say xor. it must not say the key.

the method: public. every team reads it, and that is fine.

the key: secret. it is the only thing that must be.

kerckhoffs, 1883. your browser encrypts with methods from the textbook.

one byte of key: 256 tries

what the neighbours read, the first 16 of 196 662 bytes

e5 ea 91 a7 a4 a7 a7 a7 a7 a7 91 a7 a7 a7 8f a7

looks like noise. but one byte of key means 256 possible keys, so a program tries them all:

key 00	e5 ea 91 a7 a4 a7 ...	not a file anyone knows
key 01	e4 eb 90 a6 a5 a6 ...	not a file anyone knows
key 02	e7 e8 93 a5 a6 a5 ...	not a file anyone knows
⋮		
key a7	42 4d 36 00 03 00 ...	"BM": the header of a bitmap. the parrot follows.
⋮		
key ff	1a 15 6e 58 5b 58 ...	not a file anyone knows

exactly one of the 256 gives a valid header. found in a blink.

looking like noise is not the same as being noise.

why longer is better

key length	possible keys	trying them all, a billion a second
1 byte	256	instant
2 bytes	65 536	instant
4 bytes	4 294 967 296	4 seconds
8 bytes	18 446 744 073 709 551 616	585 years
16 bytes, 128 bit	$3.4 \cdot 10^{38}$	longer than the universe has existed

every byte of key multiplies the work by 256. sixteen bytes are not sixteen times harder than one, they are out of reach.

every byte of key multiplies the work by 256. that is why keys are long.

a billion tries a second: the order of one graphics card

how long is long enough

your password is a key

the password		possible passwords	trying them all
8 lowercase letters	26^8	$2 \cdot 10^{11}$	3.5 minutes
8 mixed: upper, lower, digits, symbols	95^8	$7 \cdot 10^{15}$	77 days
12 lowercase letters	26^{12}	$1 \cdot 10^{17}$	3 years
12 mixed	95^{12}	$5 \cdot 10^{23}$	17 million years

a word from a list: password123, summer2026, your cat's name under a second, at any length

attackers do not start at aaaaaaaaa. they start with lists: leaked passwords, dictionary words, names, years. a list is tried in seconds.

length beats symbols. and a word from a list is no key at all.

same billion tries a second · check your own on haveibeenpwned.com

how long is long enough

three bytes of key, and the parrot is still there



raw

196 608 bytes, what you want to send



xor with 5a c3 96, repeated

what the neighbours display

encrypted is not the same as unrecognisable.

why the pattern survives

three points of the black background

in	00	00	00	00	00	00	00	00	00
key	5a	c3	96	5a	c3	96	5a	c3	96
out	5a	c3	96	5a	c3	96	5a	c3	96

same in, same key: same out

three points of the plumage

in	e0	29	13	da	0d	0d	d9	08	0b
key	5a	c3	96	5a	c3	96	5a	c3	96
out	ba	ea	85	80	ce	9b	83	cb	9d

different in: different out

the background turns into one colour, the plumage stays varied. every byte is replaced, always the same way, and the picture keeps its shape.

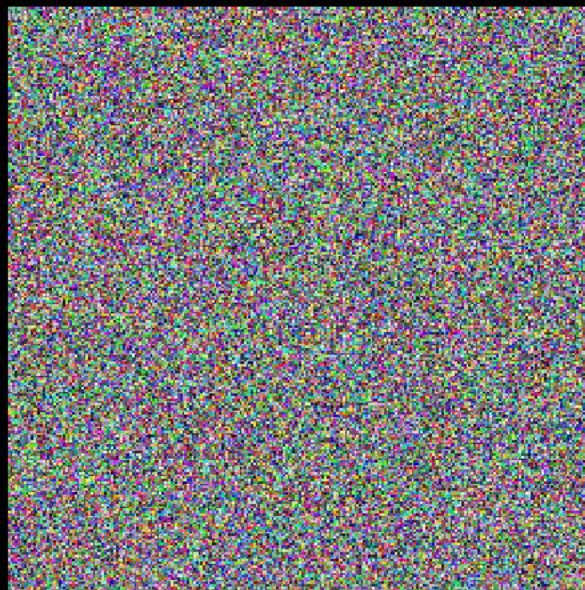
a short key treats equal bytes equally. the picture keeps its shape.

a key as long as the picture, used once



three bytes, repeated

65 536 times



196 608 random bytes, used once

nothing repeats, nothing shows

now it is noise. provably. and it has a price.

real files, 256 × 256 points · parrot: ai-generated (gpt-image-2)

how long is long enough

how the key gets to your partner

send the key over the link, in the clear

→ the neighbours read it, and everything after it

send the key over the link, encrypted

→ encrypted with which key?
the same question, one level down

hand it over where nobody listens

→ the same table, a slip of paper. done.

in the course the second path is easy: you sit next to each other. between two computers that never met, it is the hard part of the whole field.

the key must travel. and it must not travel on the link.

strangers: a padlock instead of a slip of paper



1 the shop sends an open padlock. anyone may snap it shut; only the shop's key opens it.

a lock anyone can shut and only one can open: the secret travels locked, the key never travels.

strangers: a padlock instead of a slip of paper



1 the shop sends an open padlock. anyone may snap it shut; only the shop's key opens it.

2 the browser puts a fresh secret in, snaps it shut, sends it back. nobody on the way can open it, not even the browser any more.

a lock anyone can shut and only one can open: the secret travels locked, the key never travels.

strangers: a padlock instead of a slip of paper



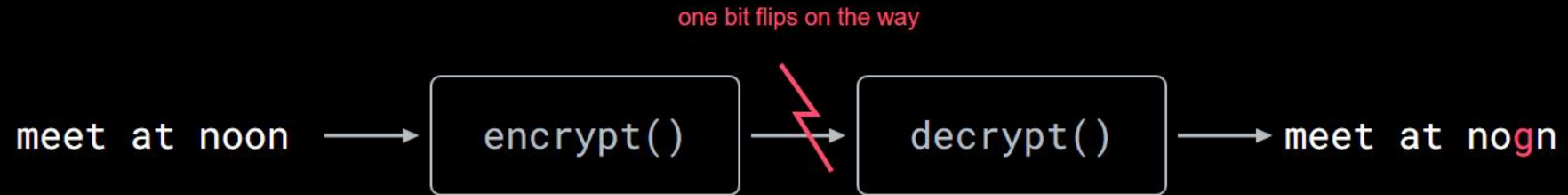
- 1 the shop sends an open padlock. anyone may snap it shut; only the shop's key opens it.
- 2 the browser puts a fresh secret in, snaps it shut, sends it back. nobody on the way can open it, not even the browser any more.
- 3 the shop opens it. now both know the secret, the line saw a lock and never a key, and from here on they xor with the secret as before.

a lock anyone can shut and only one can open: the secret travels locked, the key never travels.

part 4

what a key does not do

flipped bits go straight through



the key flips the bits back that it flipped. the one bit the link flipped, it flips too: 6f becomes 67, an o becomes a g, and nothing says so.

a checksum over the sent bytes notices: mismatch, send that frame again.

a key keeps others from reading. it does not keep bits from changing.

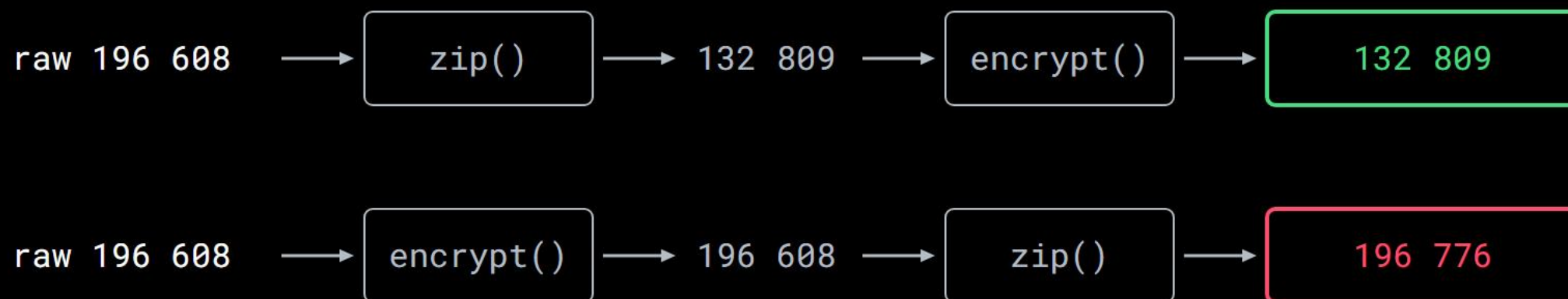
what stays in the clear



the frame does its job in the clear. only the content is locked, and the checksum is computed over the locked bytes.

encrypt the content. the frame around it has a job to do.

pack first, then lock



encrypted bytes have no redundancy left: the packer finds nothing and adds its table. the wrong order costs 64 kilobytes.

encrypted bytes have no redundancy left. pack before you lock.

hide it in the parrot



the parrot

196 608 bytes

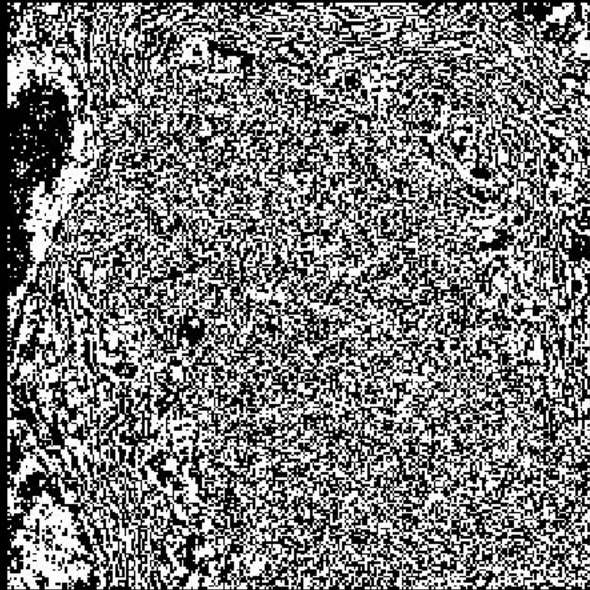


the parrot + 24 576 characters of
hidden text

196 608 bytes, 98 556 of them changed by exactly 1

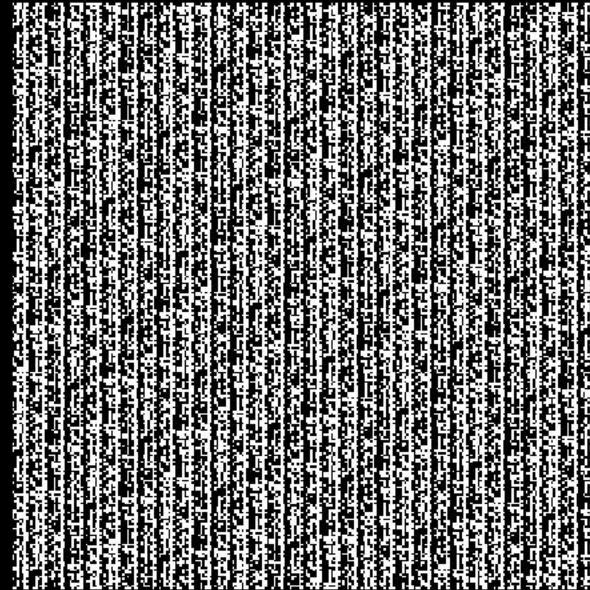
not scrambled. hidden, in the bit nobody looks at.

the lowest bit, made visible



lowest bit of red, original

noise with traces of the picture



the same bit with the text inside

the stripes of ascii, to the naked eye

$256 \times 256 \times 3 = 196\,608$ values $\cdot$ 1 bit each $\div 8 = 24\,576$ characters

whoever knows where to look reads it in a blink. hidden is not secret.

hidden is not secret. secret is not hidden.

encryption: everyone sees a message, nobody can read it.

steganography: nobody sees a message, anyone who looks can read it.

want both? encrypt first, then hide. the finder reads noise, and noise was there before.

today

- 1 light is public. the key is what protects the content.
- 2 xor with the key, twice, and the message is back.
- 3 the method may be known. the key must not, and it must be long.
- 4 the key needs a second path: a table, or a padlock.
- 5 a key stops reading, not changing: keep the checksum. pack first, then lock.

readable by everyone.
understood by one.

